01 QUESTION ONE · JAVA & OOPS

What is Java? Write concepts of OOPS which used in Java.

4 A4 PAGES
JAVA PROGRAM ON PAGE 4

WHAT IS JAVA?

Java is a high-level, object-oriented and platform-independent programming language used to build everything from small desktop tools to large enterprise systems. It was developed by **James Gosling** and first released by **Sun Microsystems in 1995**. Gosling wanted a language that could be written once and run on any machine, and that idea became Java's famous motto — **"Write Once, Run Anywhere"** (WORA). The project was originally named **"Oak"** after the tree outside his office, and was renamed Java after a popular brand of coffee.

The most important thing about Java is what happens after compilation. Java source is not compiled straight into machine language; the compiler produces a special file called **bytecode**, which any Java environment can run.

Simple	Object-Oriented	Platform-Independent
Robust	Secure	Multithreaded
Portable	High Performance	Distributed

FIG. 1 — THE QUALITIES JAVA IS KNOWN FOR, SET OUT AS A GRID. THE MIDDLE ROW HOLDS THE ONES A PROGRAMMER FEELS EVERY DAY: THE PROGRAM DOES NOT CRASH, THE BYTECODE IS VERIFIED BEFORE IT RUNS, AND SEVERAL TASKS CAN RUN TOGETHER.

JVM, JRE AND JDK

The three names are nested inside one another.

JVM — Java Virtual Machine. It loads the `.class` file, checks the bytecode for safety and executes it. Because the bytecode is the same everywhere, the program behaves identically on every operating system.

JRE — Java Runtime Environment. It is the JVM plus the class libraries a running program needs. It is enough to *run* a Java program, but not to build one.

JDK — Java Development Kit. It is the JRE plus the compiler `javac`, the debugger and the documentation tool. It is needed to *write* a Java program.

So a student uses the **JDK**, the compiler produces bytecode, and the **JRE** on any machine runs it through the **JVM**.

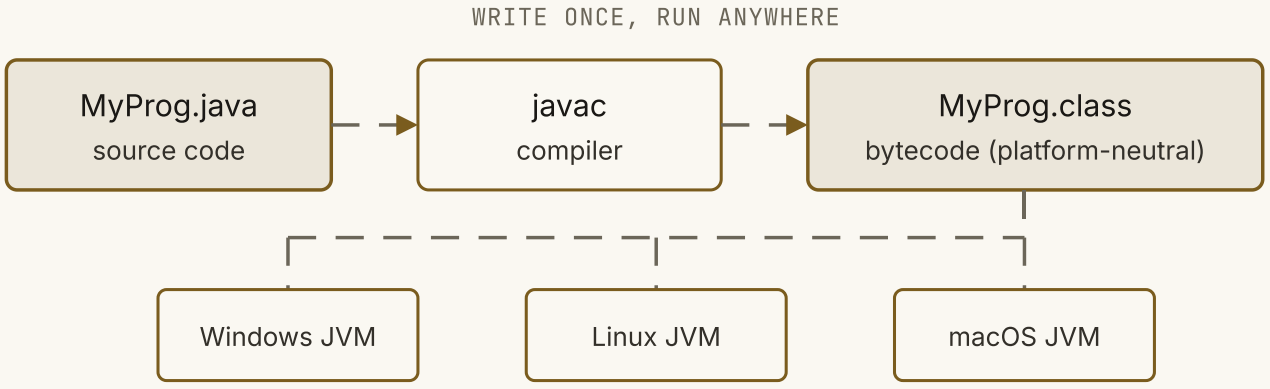


FIG. 2 — THE LIFE OF A JAVA PROGRAM. THE SAME **.CLASS** BYTECODE IS TAKEN BY THE JVM OF ANY OPERATING SYSTEM, SO THE SOURCE HAS TO BE WRITTEN ONLY ONCE.

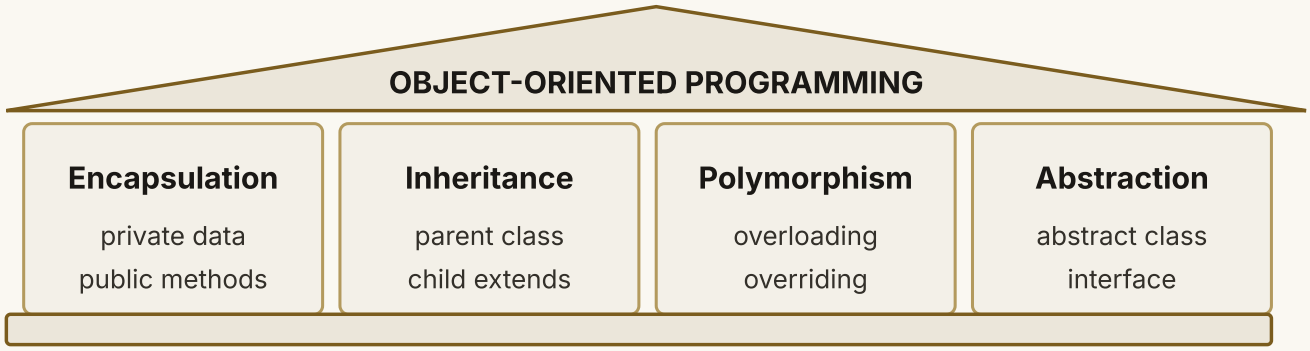
CONCEPTS OF OOPS USED IN JAVA

1. Encapsulation means wrapping data and the methods that work on it inside one class and hiding the data behind `private`. Outside code must use the public methods, where the values can be checked before they are accepted.

2. Inheritance lets a new class take the data and methods of an existing class. The old class is the parent or superclass and the new one is the child or subclass, written with the `extends` keyword. This removes repeated code.

3. Polymorphism means "many forms". In **overloading** the same method name is used with different parameter types; in **overriding** a child class gives its own body to a method it has inherited.

4. Abstraction means showing only the essential details. It is written through **abstract classes** and **interfaces**, where only the declaration is given and the real code is supplied later.



ALL FOUR TOGETHER MAKE A JAVA PROGRAM OBJECT-ORIENTED

FIG. 3 — THE FOUR PILLARS, DRAWN AS FOUR PILLARS. EACH ONE SOLVES A DIFFERENT PROBLEM, AND A WELL-WRITTEN JAVA CLASS USUALLY SHOWS ALL FOUR AT WORK AT THE SAME TIME.

JAVA PROGRAM

ONE PROGRAM THAT USES ALL FOUR OOP CONCEPTS TOGETHER. `ANIMAL` SHOWS ABSTRACTION, BECAUSE IT DECLARES `SOUND()` WITHOUT GIVING IT A BODY. `DOG` SHOWS INHERITANCE WITH `EXTENDS`, AND IT SHOWS POLYMORPHISM BECAUSE `@OVERRIDE` GIVES THE INHERITED METHOD ITS OWN BODY. THE VARIABLE `REF` IS DECLARED AS AN `ANIMAL` YET HOLDS A `DOG`, AND JAVA CALLS THE RIGHT VERSION AT RUN TIME. THE PRIVATE FIELD `NAME` IS THE ENCAPSULATED DATA, REACHED ONLY THROUGH `GETNAME()`.

OOPSDEMO.JAVA

```java
class Animal {                      // abstraction
    void sound() { System.out.println("..."); }
}
class Dog extends Animal {          // inheritance
    @Override void sound() { System.out.println("Bhau Bhau"); }
}
public class OopsDemo {              // encapsulation + polymorphism
    private String name = "Rex";    // data hidden
    public String getName() { return name; }
    public static void main(String[] a) {
        Animal ref = new Dog();      // ref is Animal, object is Dog
        ref.sound();                // polymorphism at work
        OopsDemo d = new OopsDemo();
        System.out.println(d.getName());
    }
}
```

Output

OUTPUT

```
Bhau Bhau
Rex
```

CONCLUSION

Java combines all four OOP ideas, and its bytecode is what makes the language platform independent.

02 QUESTION TWO · ENCAPSULATION

Write one example for encapsulation and define each function.

4 A4 PAGES
PROGRAM ON PAGE 3, FUNCTIONS ON PAGE 4

WHAT IS ENCAPSULATION?

Encapsulation is the process of wrapping data and the code that works on it together in a single unit called a **class**, and then keeping the data **private** so that no outside code can change it directly. The only way in is through the **public methods** of the class, and each of those methods can check the value first. This is also called **data hiding**.

In Java the private data members are written with `private`, and the methods that open the class to the outside world with `public`.

Advantages of Encapsulation

Protection: the data cannot be spoiled, because no outside code may touch it. **Validation:** a setter can refuse a wrong value, for example a negative balance. **Loose coupling:** the class can change its internal design later without breaking anybody, because everyone uses only the public methods. **Reuse:** a correct, tested class can be used again and again in different programs.

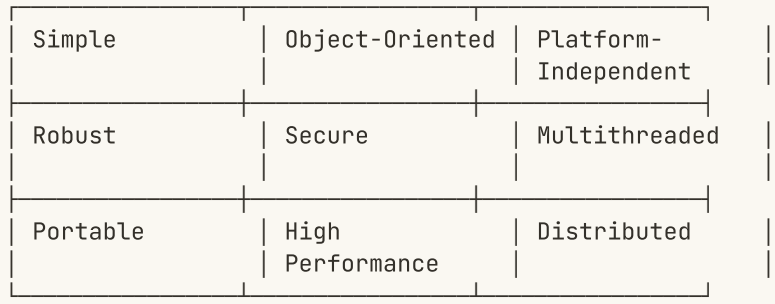


FIG. 1 — THE QUALITIES JAVA IS KNOWN FOR, SET OUT AS A GRID. THE MIDDLE ROW HOLDS THE ONES A PROGRAMMER FEELS EVERY DAY: THE PROGRAM DOES NOT CRASH, THE BYTECODE IS VERIFIED BEFORE IT RUNS, AND SEVERAL TASKS CAN RUN TOGETHER.

EXAMPLE — THE BANKACCOUNT CLASS

To see encapsulation in practice we take a **BankAccount**. The **balance** is important and sensitive data; if anybody could change it directly the whole banking system could be damaged. So the balance is made **private**, and it is reached only through public methods.

MEMBER	PURPOSE
private String owner	Name of the holder (hidden)
private double balance	Money in the account (hidden)
getBalance()	Returns the balance for reading
deposit(a)	Adds money, refusing a negative amount
withdraw(a)	Removes money only if it is available

The two data members are private while every function that touches them is public. That single line is encapsulation: **private data, public methods**.

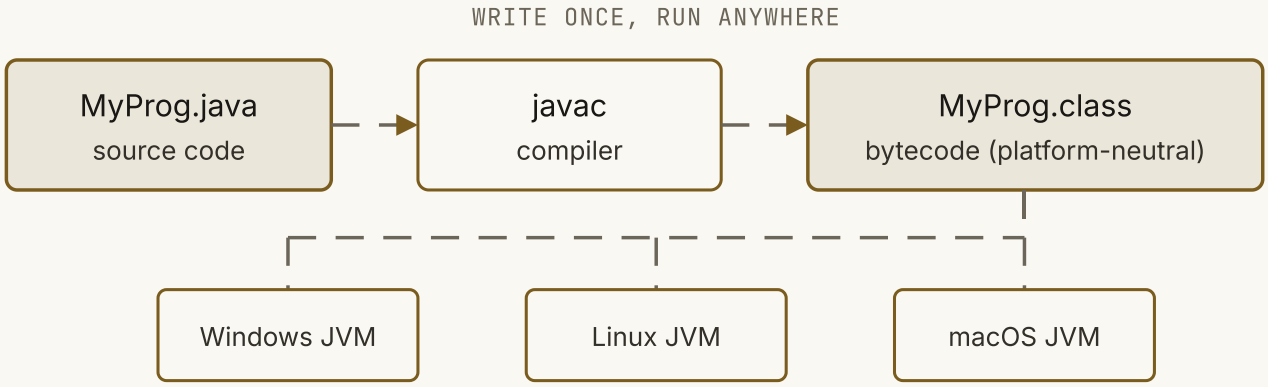


FIG. 2 — THE LIFE OF A JAVA PROGRAM. THE SAME **.CLASS** BYTECODE IS TAKEN BY THE JVM OF ANY OPERATING SYSTEM, SO THE SOURCE HAS TO BE WRITTEN ONLY ONCE.

JAVA PROGRAM

THE BALANCE IS PRIVATE AND CAN BE REACHED ONLY THROUGH PUBLIC METHODS. NOTICE THAT NO CLASS OUTSIDE THIS ONE CAN WRITE TO `BALANCE` DIRECTLY, EVEN FROM THE SAME FILE, BECAUSE IT IS DECLARED `PRIVATE`. EVERY CHANGE MUST PASS THROUGH A METHOD, AND EACH METHOD CHECKS ITS OWN ARGUMENT FIRST, SO AN IMPOSSIBLE VALUE CAN NEVER REACH THE DATA.

BANKACCOUNT.JAVA

```java
class BankAccount {
    private String owner;        // hidden data
    private double balance;

    BankAccount(String o, double b) { owner = o; balance = b; }

    public double getBalance() { return balance; }

    public void setBalance(double b) {
        if (b ⩾ 0) balance = b;   // validated
    }
    public void deposit(double a) {
        if (a > 0) balance += a;
    }
    public void withdraw(double a) {
        if (a > 0 && a ⩽ balance) balance -= a;
        else System.out.println("Not enough money");
    }
    public void show() {
        System.out.println(owner + " : " + balance);
    }
}
```

Definition of Each Function

getBalance() returns the hidden balance so it can be read.
setBalance(b) accepts a new balance only if it is not negative.
deposit(a) adds money only when the amount is positive.
withdraw(a) removes money only when it is available, otherwise it prints a warning.
show() prints the holder — name and the balance together, and is the only method allowed to display them.

Running the program shows encapsulation doing its job. A deposit of 1500 is accepted and a withdrawal of 2000 succeeds, but the attempt to take out 9000 is refused with a printed warning, because 9000 is more than the balance. If the balance had been a public field, any line of code could have set it to a wrong or negative value, and nothing would have complained.

MAIN

```
public static void main(String[] args) {
    BankAccount acc = new BankAccount("Rahul", 5000);
    acc.deposit(1500);
    acc.withdraw(9000);     // refused
    acc.withdraw(2000);
    acc.show();
}
```

Output

OUTPUT

```
Not enough money
Rahul : 4500
```

CONCLUSION

Encapsulation keeps the data safe and forces every change to pass through a method that can check it.

03 QUESTION THREE · CLASS ACCESS

What is the difference between private and public class?

4 A4 PAGES
TABLE + PROGRAM ON PAGE 4

WHAT IS A CLASS AND WHAT IS AN ACCESS MODIFIER?

A **class** is a blueprint for objects. It describes what the object will hold (its *data members*) and what it will be able to do (its *methods*). An **access modifier** is the keyword written before a class or a member to decide **who is allowed to use it**.

Java provides four access modifiers.

MODIFIER	VISIBLE INSIDE THE CLASS	VISIBLE IN THE PACKAGE	VISIBLE OUTSIDE
private	Yes	No	No
default	Yes	Yes	No
protected	Yes	Yes	Only by a subclass
public	Yes	Yes	Yes, everywhere

Why the difference matters: a public class can be used by any program that has the file, while a private class is locked inside its own file. The modifier decides how far a class can travel.

The keywords also control what a **subclass** may do. A `protected` member is open to the class that declares it, to every class in the same package, and to any subclass living anywhere, but to nobody else. This is why a protected field is usually the one a parent class wants its children to use and its children alone.

Two rules complete the picture. A member declared `public` in one class is certainly usable from another. But a member with *no* keyword is not automatically public; it is **package-private**, which is why the words *public member* must be used carefully.

PUBLIC CLASS

A class declared `public` may be used by **any other class**, anywhere in the whole project, and a Java file may be imported by other files through `import`.

Characteristics of a Public Class

It must be saved in a file whose **name is exactly the same** as the class name, including the capital letters — `public class Student` must live in `Student.java`. Only **one public class** is allowed in one file, although that file may contain any number of non-public classes beside it. Because everybody can reach it, its methods must also be declared `public` for the outside code to use them.

Other Access Modifiers for Reference

protected is like public but opens only to the classes that lie in the same package or that extend this class. **default**, used when no keyword is written, opens only inside the same package. **private** opens to nothing outside the class itself.

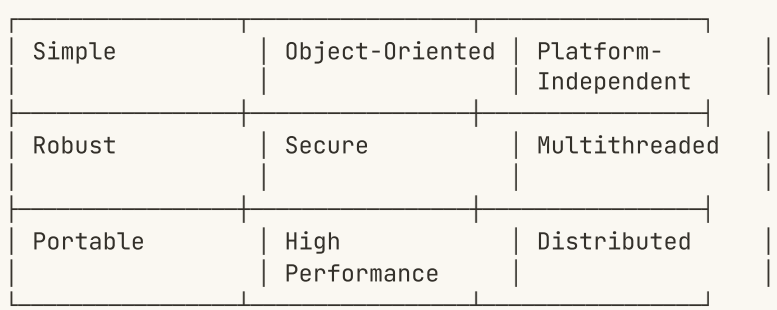


FIG. 1 — THE QUALITIES JAVA IS KNOWN FOR, SET OUT AS A GRID. THE MIDDLE ROW HOLDS THE ONES A PROGRAMMER FEELS EVERY DAY: THE PROGRAM DOES NOT CRASH, THE BYTECODE IS VERIFIED BEFORE IT RUNS, AND SEVERAL TASKS CAN RUN TOGETHER.

PRIVATE CLASS

A class declared `private` is accessible **only inside the class in which it is defined**, and not from outside at all. In Java such a class works as a **nested class**, also called an inner class, so it is used only inside its **enclosing (outer) class** and no outside class can create an object of it.

Characteristics of a Private Class

We use a private class when we want the whole implementation of some part of the program to stay hidden. If `Outer` contains a `private class Helper`, then only `Outer` may use it; nothing outside it can even name it. Because it is locked away, the helper can be changed at any time without breaking the rest of the program. It also keeps the public face of a class small, which makes the class easier to use, to test and to improve.

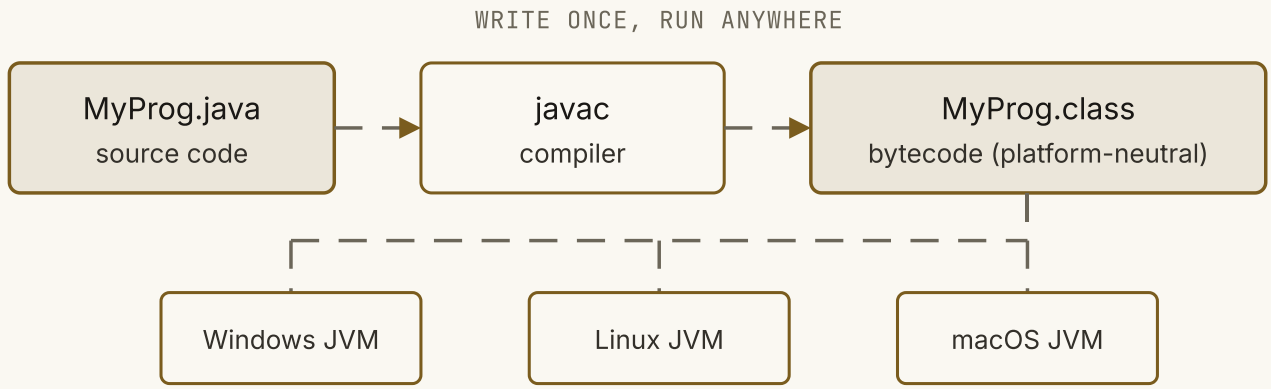


FIG. 2 — THE LIFE OF A JAVA PROGRAM. THE SAME **.CLASS** BYTECODE IS TAKEN BY THE JVM OF ANY OPERATING SYSTEM, SO THE SOURCE HAS TO BE WRITTEN ONLY ONCE.

POINT	PUBLIC CLASS	PRIVATE CLASS
Visibility	Everywhere in the project	Only inside the outer class
Use	Reused by many programs	Hidden helper of one class

JAVA PROGRAM

DEMO.JAVA

The table above shows the whole difference in two lines: a public class is reached from anywhere, and a private class is reached from nowhere except the class that holds it.

```java
public class Demo {                  // public class
    private class Helper {           // private, locked inside
        void work() { System.out.println("helping"); }
    }
    public static void main(String[] a) {
        Demo d = new Demo();
        d.new Helper().work();       // only Demo can do this
    }
}
```

Output

OUTPUT

```
helping
```

THE OUTER CLASS IS DECLARED `PUBLIC`, SO `DEMO` MAY BE USED BY ANY OTHER PROGRAM. THE INNER CLASS IS DECLARED `PRIVATE`, SO `HELPER` CANNOT BE NAMED, LET ALONE CREATED, BY ANYTHING OUTSIDE `DEMO`. BECAUSE AN INNER CLASS IS TIED TO ONE OBJECT OF THE OUTER CLASS, IT IS REACHED AS `D.NEW HELPER()` — THE INSTANCE `D` FIRST, AND THEN A NEW `HELPER` BELONGING TO IT.